GIT-APE - TRACK 3 - PLATFORM ENGINEERING

Platform Engineering

CI/CD, headless mode, multi-environment, policy, lifecycle

90 minutes hands-on for DevOps, SRE, platform engineers

OIDC zero secrets | Headless mode | Multi-env | CIS / NIST



AI-powered cloud deployment

9 slides | 90 minutes | Platform

CI/CD Architecture

PR-based deployments with zero stored secrets

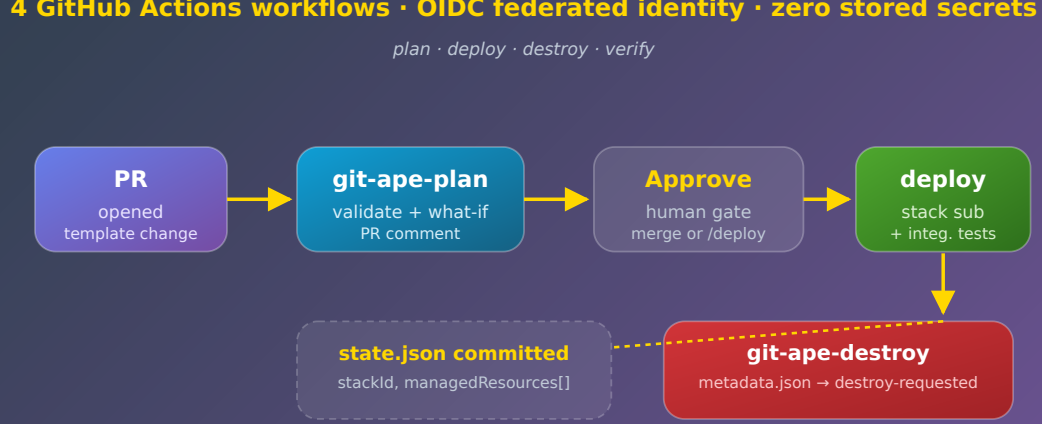
Four GitHub Actions workflows

1. `git-ape-plan.yml` - validates PR, posts plan as comment
2. `git-ape-deploy.yml` - deploys on merge or `/deploy`
3. `git-ape-destroy.yml` - tears down when `metadata.json` flips to `destroy-requested`
4. `git-ape-verify.yml` - manual OIDC / RBAC check

OIDC federated identity

Azure Deployment Stacks

Pipeline flow



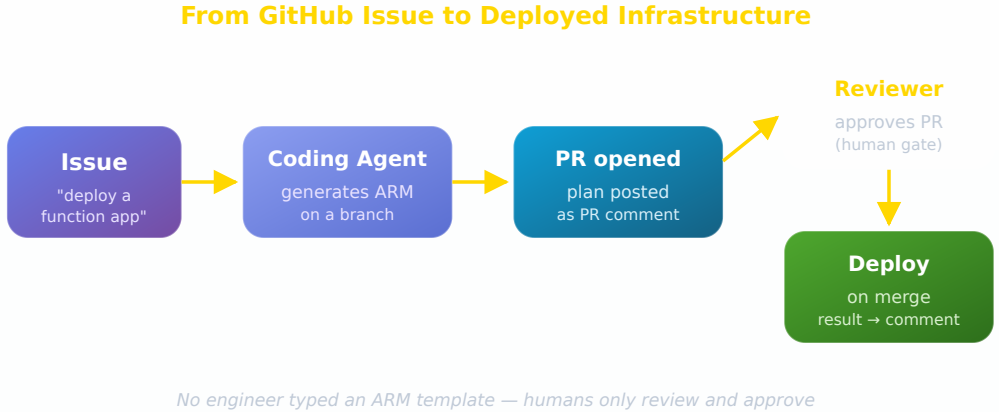
Headless Mode

GitHub Issue to deployed infrastructure - no engineer typing

The flow

1. File an issue describing what you need
2. **Copilot Coding Agent** generates ARM template on a branch
3. PR opened with architecture diagram and plan
4. `git-ape-plan` validates and posts the plan
5. Human reviewer approves the PR
6. Deploy on merge - results posted back as PR comment

From issue to deployment



No engineer typed an ARM template — humans only review and approve

Multi-Environment Strategy

Dev to staging to prod with isolation

Isolation model

- Separate **Azure subscriptions** per environment
- Environment-specific **parameter files**
- **GitHub environments** with protection rules
- OIDC credentials **scoped per environment**
- **Promotion requires PR approval** at each stage

Per-environment scoping

dev

Automatic deploy on merge - sandbox subscription, B1 SKUs

staging

One approver - staging subscription, S2 SKUs

prod

Two approvers plus wait timer - prod subscription, P1v3 plus geo-redundancy

Policy and Compliance

Azure Policy assessment with CIS, NIST, or custom initiatives

What is assessed

- **Frameworks:** CIS Azure Foundations v3.0, NIST SP 800-53 Rev 5
- **Categories:** identity, networking, storage, compute, monitoring, tagging
- **Per-resource** policy recommendations
- **Audit vs Deny** enforcement modes

Advisory gate | Pre-deploy

Gate behaviour

Advisory

Surfaces findings without halting deployment - safe for adoption

Audit then Deny

Start audit-only, graduate to Deny for Critical policies once baseline is clean

Output artifacts: `policy-assessment.md` and `policy-recommendations.json` per deployment.

IaC Export

Bring existing Azure resources under Git-Ape management

What it does

- **Export live Azure resources** to ARM templates
- **Reverse-engineer** existing infrastructure
- Bring **legacy deployments** under management
- **Compare exported template** against live state for drift

No greenfield required

Drift baseline

Typical adoption path

1. Export

Point the IaC Exporter at an existing resource group or subscription

2. Review

Inspect the generated template, refactor parameters, add tags

3. Adopt

Re-deploy as a stack - drift detection runs on every subsequent change

Lifecycle Management

Create -> Deploy -> Monitor -> Teardown

State progression

- `planning` -> template generated, not yet deployed
- `deployed` -> stack created, state.json committed
- `drift-detected` -> live state diverged from template
- `reconciled` -> drift resolved (template or live updated)
- `destroyed` -> stack deleted, audit trail preserved

Teardown is PR-driven

1. Open PR

Set `metadata.json` status to `destroy-requested`

2. Approve

Human gate for destructive action - required reviewers on `azure-destroy` env

3. Merge -> auto-delete

`az stack sub delete --action-on-unmanage deleteAll` - no orphans, idempotent

Lab Roadmap

Six labs across 90 minutes

Lab	Time	What you do
1. CI/CD Pipeline	20 min	PR-based plan and deploy workflow
2. Headless Mode	15 min	Issue triggers auto-PR and deploy
3. Multi-Environment	15 min	Dev to staging promotion
4. Policy Compliance	15 min	CIS / NIST assessment
5. IaC Export	10 min	Export existing resources
6. Destroy Lifecycle	5 min	PR-driven teardown

Labs 1, 2, and 6 are the highest-impact demos. If time slips, abbreviate Lab 4 or Lab 5.

Let's Go!

Verify onboarding, then start Lab 1

1. Confirm OIDC

From Track 2 Lab 1
or onboarding playbook

Federated credential + RBAC

2. Verify

Run `git-ape-verify.yml`
manual workflow
dispatch

Health check, no deployment

3. Lab 1

CI/CD pipeline
20 minutes

Foundation for the rest

Prerequisite: Completed Track 2 onboarding or equivalent Git-Ape experience.